Week 3, Day 2: Spark Memory Management

AWS Data Engineering with PySpark - Theory Guide

Sachin Chandrashekhar - Data Engineering Hub

Follow us on social media for exciting updates and exclusive content!

YouTube: https://www.youtube.com/@Men_torSachin

Instagram: https://www.instagram.com/mentorsachin/

LinkedIn: https://www.linkedin.com/in/mentorsachin/

X (Twitter): https://x.com/mentorsachin_

Website 1: https://sachinchandrashekhar.com

Website 2: https://dataengineeringhub.in

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

CRITICAL INTERVIEW TOPIC

Memory management appears in 70%+ of Spark interviews. This is complex material that requires 2-3 hours to fully internalize. Take detailed notes and explain concepts in your own words.

Overview

Understanding Spark memory management is essential for:

- • Optimizing Spark application performance
- • Avoiding Out Of Memory (OOM) errors
- • Properly sizing executors and drivers
- • Answering interview questions confidently
- • Troubleshooting production issues

Memory Request Example

Let's start with a typical spark-submit command:

```
spark-submit \   --deploy-mode cluster \   --driver-memory 4g \
--driver-cores 2 \   --executor-memory 8g \   --num-executors 2 \
--executor-cores 3 \   my_spark_job.py
```

What you requested:

- • Driver: 4 GB memory, 2 cores
- • 2 Executors: 8 GB memory each, 3 cores each
- • Total executor memory (naive): 2 × 8 GB = 16 GB

But here's the catch:

When you request 8 GB per executor, you don't actually get just 8 GB from the cluster. You get MORE!

Overhead Memory (The Hidden Cost)

Formula:

`Overhead Memory = MAX(384 MB, 10% of executor memory)`

Configuration: `spark.executor.memoryOverhead`

Example Calculation (8 GB executor):

- • You request: 8 GB
- • 10% of 8 GB = 800 MB
- • MAX(384 MB, 800 MB) = 800 MB
- • Overhead memory: 800 MB
- **• Total requested from cluster: 8 GB + 800 MB = 8.8 GB**

What is overhead memory used for?

- • JVM internal memory structures
- • Thread stacks

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

- • Native libraries
- • Off-heap allocations by libraries
- • Any non-JVM memory usage

Why This Matters for Cluster Sizing:

Scenario: Your cluster has 24 GB total memory

WRONG CALCULATION:

- • Think: 3 executors × 8 GB = 24 GB (fits perfectly!)
- • Reality: 3 executors × 8.8 GB = 26.4 GB
- • Result: REQUEST WILL FAIL (not enough memory)

CORRECT CALCULATION:

- • Available: 24 GB
- • Per executor (including overhead): 8.8 GB
- • Maximum executors: 24 / 8.8 = 2.72 → 2 executors
- • Or use smaller executors: 7 GB each → 7.7 GB total → 3 executors fit!

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

Inside the 8 GB JVM Heap Memory

JVM (Java Virtual Machine):

- • Spark is written in Scala (runs on JVM)
- • PySpark is a wrapper around Scala/Java code
- • When you submit PySpark code, Scala executes underneath
- • JVM = Runtime environment for Spark code

Memory Breakdown (8 GB Executor):

Step 1: Reserved Memory (Fixed)

- • Reserved Memory: 300 MB (ALWAYS FIXED)
- • Used for: Spark engine internal operations
- • Not available for your application
- • Remains 300 MB regardless of executor size
- **• Usable Memory: 8 GB - 300 MB = 7.7 GB**

Note on Reserved Memory:

While 300 MB is the standard minimum, technically reserved memory is calculated as: executor memory × (1 - spark.memory.fraction). For most configurations with spark.memory.fraction=0.6, this results in the 300 MB we commonly see.

Step 2: User Memory (40% of Usable)

- • User Memory: 40% of 7.7 GB = 3.08 GB
- • Configuration: spark.memory.fraction (default 0.6 means 40% for user)

• Used for:

  - 1. User-written RDD operations (if any)
  - 2. Python data structures (lists, dicts, sets)
  - 3. User-defined objects in code

CRITICAL DISTINCTION:

DataFrame operations (df.join(), df.groupBy(), df.sort()) use EXECUTION MEMORY, NOT user memory!

User memory is ONLY for:

- • Explicit RDD transformations you write (rare nowadays)
- • Python variables, lists, dictionaries in your code

Since we primarily use DataFrame API, user memory is mostly for Python data structures.

Step 3: Unified Memory (60% of Usable)

- • Unified Memory: 60% of 7.7 GB = 4.62 GB
- • Configuration: spark.memory.fraction (default 0.6)
- • Called 'unified' because boundary is flexible (explained later)

Unified Memory Split (50-50 by default):

A) Storage Memory: 50% of unified = 2.31 GB

- • Configuration: spark.memory.storageFraction (default 0.5)
- • Used for: Cached/persisted DataFrames/RDDs
- • Example: df.cache() or df.persist()

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

- • When you cache, data stored here

B) Execution Memory: 50% of unified = 2.31 GB

- • Used for: All DataFrame transformations
- • Operations: Joins, sorts, aggregations, shuffles
- • This is where most processing happens
- • Critical for performance

Complete Memory Breakdown (8 GB Executor):

Component	Size	% of Total	Purpose
Total Request	8.8 GB	110%	From cluster
- JVM Heap	8 GB	100%	Application memory
- Overhead	800 MB	10%	JVM internals, native libraries
Reserved Memory	300 MB	3.75%	Spark engine
Usable Memory	7.7 GB	96.25%	For application
User Memory	3.08 GB	38.5%	RDD ops, Python objects
Unified Memory	4.62 GB	57.75%	Flexible between storage/execution
- Storage Memory	2.31 GB	28.9%	Cached data
- Execution Memory	2.31 GB	28.9%	Joins, sorts, shuffles, DataFrame ops

Example: 4 GB Executor Breakdown

- • Total from cluster: 4.4 GB (4 GB + 400 MB overhead)
- • JVM Heap: 4 GB
- • Reserved: 300 MB
- • Usable: 3.7 GB
- • User Memory (40%): 1.48 GB
- • Unified Memory (60%): 2.22 GB
    - - Storage: 1.11 GB
    - - Execution: 1.11 GB

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

Unified Memory: Why 'Unified'?

The 50-50 split between Storage and Execution is NOT fixed. The boundary is FLEXIBLE.

Scenario 1: Storage Needs More Memory

Initial State:

- • Storage: 2.31 GB
- • Execution: 2.31 GB
- • Total Unified: 4.62 GB

What Happens:

- • Your code caches multiple DataFrames
- • Storage needs 4 GB (more than allocated 2.31 GB)
- • Execution memory is currently IDLE (no joins/sorts running)

Storage BORROWS from Execution:

- • Storage needs 2 GB more
- • Execution has 2.31 GB available (idle)
- • Storage borrows 2 GB from Execution

New State:

- • Storage: 4.31 GB (2.31 + 2)
- • Execution: 0.31 GB (2.31 - 2)
- • Total Unified: Still 4.62 GB

KEY RULE:

Storage can borrow from Execution ONLY IF Execution memory is currently available (idle).

Scenario 2: Execution Needs Memory Back

Current State (from Scenario 1):

- • Storage: 4.31 GB (borrowed 2 GB)
- • Execution: 0.31 GB

What Happens:

- • Now your code performs a large join
- • Execution needs 1.5 GB for the join
- • Currently has only 0.31 GB

Execution EVICTS from Storage:

- • Execution needs 1.5 GB
- • Execution reclaims 1.5 GB from Storage
- • Cached data in Storage gets SPILLED to disk

New State:

- • Storage: 2.81 GB (4.31 - 1.5)
- • Execution: 1.81 GB (0.31 + 1.5)
- • Spilled to disk: Some cached partitions

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

KEY RULES:

- 1. Execution CAN evict cached data from Storage
- 2. Execution can reclaim up to its ORIGINAL threshold (2.31 GB in our example)
- 3. Execution CANNOT evict beyond the original 50-50 boundary
- 4. When evicted, cached data is spilled to LOCAL DISK (not HDFS)

Scenario 3: Execution Needs MORE Than Threshold

Current State:

- • Storage: 2.31 GB (original)
- • Execution: 2.31 GB (original)

What Happens:

- • Complex join needs 3 GB
- • Execution only has 2.31 GB
- • Cannot evict more from Storage (already at boundary)

Result: Execution Must SPILL to Disk

- • Intermediate join data spilled to disk
- • Very expensive operation (serialization/deserialization)
- • Can significantly slow down your job

Summary of Unified Memory:

- Storage can borrow from Execution (if Execution is idle)
- Execution can evict from Storage (up to original boundary) ● Execution CANNOT evict beyond original boundary
- If Execution needs more than boundary → Must spill to disk (expensive!)

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

Off-Heap Memory (Optional but Powerful)

What is Off-Heap Memory?

Off-heap memory is memory OUTSIDE the JVM heap. It must be explicitly enabled and configured.

Configuration:

```
spark.memory.offHeap.enabled = true spark.memory.offHeap.size = 1600m  #
1.6 GB
```

Typical Size:

- • 10-20% of executor memory
- • For 8 GB executor: 800 MB - 1.6 GB off-heap
- • Not included in the 8 GB (additional memory)

Why Use Off-Heap Memory?

Problem 1: Garbage Collection (GC) Overhead

What happens in JVM:

- • Spark creates many objects in JVM memory during processing
- • When memory gets low, JVM runs Garbage Collection (GC)
- • GC identifies and removes unused objects
- • GC process is TIME-CONSUMING
- • During GC, your application pauses (stop-the-world pauses)

Off-Heap Solution:

- • Off-heap memory is OUTSIDE JVM
- • No garbage collection for off-heap memory
- • No GC pauses
- • More predictable performance

Problem 2: Disk Spilling is Expensive

What happens during spilling:

- • Execution needs more memory than available
- • Spark spills intermediate data to disk
- • Requires SERIALIZATION (memory format → disk format)
- • Later requires DESERIALIZATION (disk format → memory format)
- • Serialization/Deserialization is VERY EXPENSIVE
- • Can slow down job significantly

Off-Heap Solution:

- • Spark prefers off-heap over disk spilling
- • Off-heap access is slower than JVM heap
- • BUT off-heap is MUCH FASTER than disk spilling
- • Avoids expensive serialization/deserialization

Performance Trade-Off:

Speed Ranking (Fastest to Slowest):

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

- 1. JVM Heap Memory (fastest, but subject to GC)
- 2. Off-Heap Memory (slightly slower, but NO GC)
- 3. Disk Spilling (slowest, expensive serialization)

Reality Check:

While off-heap access itself is slightly slower than JVM heap, it can make your application FASTER OVERALL by:

- • Avoiding GC pauses
- • Preventing expensive disk spilling
- • Providing more predictable performance

When to Use Off-Heap Memory:

- Use off-heap when:
  - • Spark UI shows high GC times
  - • You see frequent disk spilling in logs
  - • Job performance is unpredictable
  - • Working with very large data structures
  - • Need more predictable latencies

- May not need off-heap when:
  - • Small to medium-sized executors (< 4 GB)
  - • GC times are already low
  - • No disk spilling observed
  - • Jobs complete quickly without issues

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

Minimum Executor Memory

Formula:

```
Minimum Executor Memory = Reserved Memory × 1.5 = 300 MB × 1.5 = 450 MB
```

Why 450 MB minimum?

- • Reserved memory is 300 MB (fixed)
- • This leaves very little usable memory
- • Spark requires at least 1.5× reserved for stable operation
- • Cannot create executor smaller than 450 MB

Practical Impact:

If you try: --executor-memory 400m → Spark will error out!

Verifying Memory in Spark UI

Common Confusion:

You request 4 GB executor, but Spark UI shows 2 GB storage memory. Why?

Answer:

Spark UI Executors tab shows UNIFIED MEMORY (not total executor memory).

Calculation for 4 GB executor:

- • Total: 4 GB
- • Reserved: 300 MB
- • Usable: 3.7 GB
- • User memory (40%): 1.48 GB (not shown in UI)
- • Unified memory (60%): 2.22 GB
- **• UI displays: ~2 GB (rounded unified memory)**

What Spark UI Actually Shows:

- • Storage Memory: Maximum it can grow to (including borrowing)
- • This is why 1 GB storage can show as 2 GB
- • The 2 GB is the MAX after borrowing from execution
- • Initial storage: 1 GB
- • Can borrow: 1 GB from execution
- • Maximum: 2 GB (displayed in UI)

Interview Preparation Tips

What You MUST Know:

- 1. Overhead memory formula: MAX(384 MB, 10% of executor memory)
- 2. Reserved memory: 300 MB (fixed minimum)
- 3. Memory split: User (40%), Unified (60%)
- 4. Unified split: Storage (50%), Execution (50%)
- 5. CRITICAL: DataFrame operations use execution memory, NOT user memory

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution

- 6. Unified memory is flexible (borrowing and eviction)
- 7. Storage can borrow from execution (if idle)
- 8. Execution can evict from storage (up to boundary)
- 9. Off-heap avoids GC and disk spilling
- 10. Minimum executor: 450 MB

Be Ready to Calculate:

For any executor size (2 GB, 4 GB, 8 GB, 16 GB):

- • Overhead memory
- • Reserved memory
- • Usable memory
- • User memory
- • Unified memory (storage + execution)

Practice Drawing:

- • Memory breakdown diagram from memory
- • Show all components with sizes
- • Explain borrowing and eviction process
- • Be able to do it on a whiteboard in 3-5 minutes

Key Takeaway: Memory management is complex but follows clear rules. Master the formulas, understand unified memory behavior, and know when to use off-heap. This knowledge is essential for both interviews and production work!

Data Engineering Hub | Week 3, Day 2
© 2025 All Rights Reserved

© Sachin Chandrashekhar. All Rights Reserved Worldwide

Not For Distribution